

Project Plan: Cloud-Based Hospital Management System (Option C)

Project Objective: To design, implement, and deploy a scalable, secure, cloud-native Hospital Management System, fulfilling all requirements of our Cloud Computing assignment.

Proposed Tech Stack:

- **Frontend:** React.js (with Tailwind CSS for UI)
- **Backend:** Java Spring Boot (REST APIs, Spring Security)
- **Database:** MySQL (Cloud SQL / AWS RDS)
- **Storage (Medical Images):** AWS S3 or Azure Blob Storage
- **Virtualization/Deployment:** Docker (Containerized Microservices) deployed on AWS/Azure Free Tier.

Proposed Team Roles & Responsibilities

(Note: We can adjust these based on everyone's strengths. The assignment requires clearly defined responsibilities for peer evaluation.)

- **Member 1: Cloud Architect & Technical Writer**
 - Handles Part 1 (Conceptual Analysis) & Part 2 (Infrastructure Design).
 - Draws the professional architecture diagram and calculates the basic cost estimation.
 - Takes charge of compiling and formatting the final 4000-6000 word report.
- **Member 2: Backend Developer & DevOps**
 - Develops the Spring Boot REST APIs (Part 5 - Programming Model).
 - Handles Docker containerization and deploys the backend to the cloud VM/Container (Part 3 - Virtualization).
- **Member 3: Frontend Developer & UI/UX**
 - Develops the React web application.
 - Integrates the frontend with the backend REST APIs.
 - Ensures Role-Based Access Control (Admin, Doctor, Patient) works smoothly on the UI.

- **Member 4: Database Admin & Security/QA**

- Sets up the Cloud Database (MySQL) and Object Storage (S3) for medical imaging (Part 4).
- Configures IAM rules, security groups, and encryption (Part 6).
- Conducts basic load testing using Postman or similar tools (Part 7).

Execution Timeline (Action Plan)

Phase 1: Planning & Design (Target: Part 1 & 2 - 35 Marks)

- Define the cloud paradigm benefits, risks, and justifications for the hospital system.
- Design the complete cloud architecture (Compute, VPC, Storage, DB, Auto-scaling).
- *Output:* Finalized Architecture Diagram and Cost Estimation Table.

Phase 2: Core Development (Target: Part 4 & 5 - 35 Marks)

- Generate base code (Frontend & Backend) and set up the GitHub repository.
- Develop core modules: User Authentication (JWT), Patient Records, and Appointment Scheduling.
- Set up Cloud SQL (MySQL) and connect the Spring Boot backend.
- Implement AWS S3 (or mock interface) for X-ray/Medical Image storage.
- *Output:* Working local application and API Postman tests.

Phase 3: Cloud Deployment & Security (Target: Part 3, 6, 7 - 30 Marks)

- Write Dockerfile for frontend and backend.
- Deploy containers to the cloud (Demonstrating system/network virtualization).
- Configure IAM, Firewalls (Security Groups), and HTTPS.
- Run a basic performance/load test to identify system limits.
- *Output:* Live system accessible via a public IP/URL.

Phase 4: Final Deliverables & Polish

- Complete the Project Report (ensure APA referencing and < 20% plagiarism).
- Update the GitHub README with clear setup instructions.
- Prepare the 15-20 minute group presentation and rehearse the live demo.

Immediate Next Steps for the Group:

1. Confirm if everyone agrees with this Tech Stack (React + Spring Boot) and Option C.
2. Assign the 4 Roles mentioned above.
3. Set up the GitHub repository and invite all members.
4. Schedule a brief meeting to start drafting the Conceptual Analysis and Architecture Diagram.

```
Option C Cloud-Based Hospital Management System/
├── docker-compose.yml
├── backend/
│   ├── pom.xml
│   ├── Dockerfile
│   └── src/
│       └── main/
│           ├── java/com/hospital/system/
│           │   ├── HospitalManagementSystemApplication.java
│           │   ├── controller/           # AuthController, PatientController, etc.
│           │   ├── model/               # User, Patient, Doctor, Appointment, MedicalImage
│           │   ├── repository/          # JPA Repositories
│           │   ├── security/            # JWT & Spring Security configuration
│           │   └── service/              # Business logic & mock S3 service
│           └── resources/
│               └── application.properties # MySQL database configuration
├── frontend/
│   ├── package.json
│   ├── Dockerfile
│   ├── nginx.conf                       # Production web server config
│   ├── tailwind.config.js
│   ├── postcss.config.js
│   ├── public/
│   └── src/
│       ├── index.css
│       ├── index.js
│       ├── App.js
│       ├── components/                  # Shared UI components (Navbar, etc.)
│       ├── pages/                       # Login, Register, Dashboards
│       └── services/                    # API requests (axios wrapper)
```